DSA & Advanced DSA Roadmap

Phase 9 — Recursion

Learning Objectives

By the end of this phase, learners will be able to:

- Trace a recursion tree for a given recursive function and identify overlapping subproblems.
- Distinguish tail recursion from general recursion, and explain why tail recursion can (in principle) be optimized to avoid growing the call stack.
- Model a recursive solution as a state-space exploration: define the state, the choices available at each state, and the base case.
- Apply divide-and-conquer: split a problem into independent subproblems, solve each recursively, and combine the results.
- Solve Easy to Medium level recursion problems independently, and approach Medium/Hard divide-and-conquer problems by identifying how to split and combine.

Implementation & Internals

Core Concepts

- Call Stack — revisiting how each recursive call pushes a new stack frame, and how deep recursion risks a stack overflow (first introduced in Phase 0; now applied to more complex recursive shapes).
- Recursion Tree — visualizing a recursive function's calls as a tree, where each node is one call and children are its recursive sub-calls; counting nodes gives the time complexity.
- Tail Recursion — a recursive call that is the last operation in a function, allowing (in languages/compilers that support it) the call stack frame to be reused rather than grown.
- State Space Exploration — framing a recursive solution as moving through a space of possible states, where each recursive call represents a choice that transitions to a new state.

Implementation Exercises

- Convert a non-tail-recursive factorial function into a tail-recursive version using an accumulator parameter, and explain what changed.
- Draw the full recursion tree by hand for fib(5) using naive recursion, count the duplicate calls, and use this to justify the O(2^n) time complexity discussed in Phase 0.
- Write a recursive function to generate all subsets of a set by treating 'include element i or don't' as the state-space choice at each recursive step.

Pattern Mastery

Pattern 1: Recursive Thinking

Concepts

- Identifying the base case and recursive case for a problem that has a naturally repeating sub-structure.
- Translating a problem description directly into a recursive function signature: what does this call need to return, and what smaller version of the problem does it call itself on?

Practice Problems

- Given n, compute the number of distinct ways to climb a staircase taking 1 or 2 steps at a time.

LeetCode

- Climbing Stairs — leetcode.com/problems/climbing-stairs

Note: Climbing Stairs is intentionally revisited here (distinct from Phase 0's Fibonacci-style problems) as the canonical 'recognize the recursive structure first, optimize later' exercise; Dynamic Programming optimization of this exact problem is deferred to Phase 16.

Pattern 2: Divide & Conquer

Concepts

- Splitting a problem into independent subproblems of roughly half the size, solving each recursively, and combining the results — the same structural idea behind Merge Sort and Quick Sort from Phase 8.
- Recognizing when a problem's combine step is cheap (making divide-and-conquer efficient) versus expensive (which can erase the benefit of splitting).

Practice Problems

- Given a sorted array, build a height-balanced binary search tree by recursively choosing the middle element as the root.
- Given a string of numbers and operators, compute all possible results from adding parentheses in every possible way.

LeetCode

- Convert Sorted Array to Binary Search Tree — leetcode.com/problems/convert-sorted-array-to-binary-search-tree
- Different Ways to Add Parentheses — leetcode.com/problems/different-ways-to-add-parentheses

Pattern 3: Tree Recursion

Concepts

- Recursive functions where the recursive structure mirrors a tree shape: each call branches into two or more recursive sub-calls, even when the input isn't literally a tree data structure.
- Applying recursion directly to actual tree data structures: a function that calls itself on root.left and root.right naturally computes a property bottom-up.

Practice Problems

- Given the root of a binary tree, return its maximum depth.
- Given the root of a binary tree, invert it (swap every node's left and right children) and return the new root.

LeetCode

- Maximum Depth of Binary Tree — leetcode.com/problems/maximum-depth-of-binary-tree
- Invert Binary Tree — leetcode.com/problems/invert-binary-tree

Note: these two problems deliberately preview Phase 11 (Trees). Tree-specific traversal patterns (DFS, BFS, diameter, LCA) are covered in depth there; here they serve only as tree-recursion exercises.

Phase Assessment

Implementation Tasks

- Convert a non-tail-recursive function to tail-recursive form using an accumulator, and explain the trade-off.
- Draw a complete recursion tree by hand for a divide-and-conquer function and use it to derive the function's time complexity.
- Analyze and explain why Convert Sorted Array to Binary Search Tree always produces a height-balanced tree, tying the explanation back to the 'split in half' divide-and-conquer principle.

Recommended LeetCode Target

Easy Problems

- leetcode.com/problems/climbing-stairs
- leetcode.com/problems/maximum-depth-of-binary-tree
- leetcode.com/problems/invert-binary-tree

Medium Problems

Divide & Conquer

- leetcode.com/problems/convert-sorted-array-to-binary-search-tree
- leetcode.com/problems/different-ways-to-add-parentheses

On scope and dedup

This phase has 5 distinct problems — fewer than the 7–9 problem range of Phases 4–8 — because Recursion is a thinking tool that gets exercised heavily in the phases that follow (Backtracking, Trees, and Dynamic Programming all lean on it directly), rather than a pattern with a large independent problem set of its own. No Hard-tier problem is assigned here; Hard recursive problems (e.g. Regular Expression Matching, N-Queens) are deferred to Phase 10 (Backtracking) and Phase 16 (Dynamic Programming) where they fit more naturally. None of the 5 problems overlaps with any problem used in Phases 0–8.

Coding Challenges (Assessment Day)

- Climbing Stairs
- Maximum Depth of Binary Tree
- Convert Sorted Array to Binary Search Tree
- Different Ways to Add Parentheses

Expected Outcome

Students should be able to:

- Identify the base case and recursive case for a new problem quickly, without trial and error.
- Distinguish divide-and-conquer recursion (independent subproblems combined at the end) from simple linear recursion.
- Apply recursion directly to tree-shaped data and problems with a tree-shaped decision structure.
- Solve unseen recursion problems using pattern-based thinking rather than memorized solutions.